SLIP 1

Q1) A .Write a python program to show proper indentation using if statement. [10 marks]

Program :

```
num = int(input("Enter a number: "))

if num > 0:
    print("The number is positive.")
    print("This statement is inside the if block.")

print("Program execution completed.")
```

Sample Output

```
Enter a number: 10
The number is positive.
This statement is inside the if block.
Program execution completed.
```

Q2) B .Write a python program to implement Water Jug Problem. [20 marks]

Programme :

```
from collections import deque

def water_jug_problem(capacity1, capacity2, target):
    visited = set()
    queue = deque()

    # Initial state
    queue.append(((0, 0), []))
    visited.add((0, 0))

    while queue:
        (jug1, jug2), path = queue.popleft()

        # Check goal state
        if jug1 == target or jug2 == target:
            print("\nSolution Path:")
```

```
            for step in path:
                print(step)
            print("Goal State:", (jug1, jug2))
            return

        states = []

        # Fill Jug 1
        states.append(((capacity1, jug2), "Fill Jug
1"))

        # Fill Jug 2
        states.append(((jug1, capacity2), "Fill Jug
2"))

        # Empty Jug 1
        states.append(((0, jug2), "Empty Jug 1"))

        # Empty Jug 2
        states.append(((jug1, 0), "Empty Jug 2"))

        # Pour Jug 1 -> Jug 2
        amount = min(jug1, capacity2 - jug2)
        states.append(
            ((jug1 - amount, jug2 + amount),
             "Pour Jug 1 -> Jug 2")
        )

        # Pour Jug 2 -> Jug 1
        amount = min(jug2, capacity1 - jug1)
        states.append(
            ((jug1 + amount, jug2 - amount),
             "Pour Jug 2 -> Jug 1")
        )

        for new_state, action in states:
            if new_state not in visited:
                visited.add(new_state)
                queue.append(
                    (new_state, path + [action])
                )

    print("No solution exists.")
```

```
# Main program
capacity1 = 4
capacity2 = 3
target = 2

print("Water Jug Problem")
print("Jug 1 Capacity:", capacity1, "litres")
print("Jug 2 Capacity:", capacity2, "litres")
print("Target:", target, "litres")

water_jug_problem(capacity1, capacity2, target)
```

Sample Output

```
Water Jug Problem
Jug 1 Capacity: 4 litres
Jug 2 Capacity: 3 litres
Target: 2 litres

Solution Path:
Fill Jug 2
Pour Jug 2 -> Jug 1
Fill Jug 2
Pour Jug 2 -> Jug 1
Empty Jug 1
Pour Jug 2 -> Jug 1
Goal State: (2, 3)
```

OR

Q2)B.Write a python program to design a Model-Based Agent that maintains an internal state of room cleanliness and decides actions based on both current perception and stored memory.

```
# Model-Based Agent for Room Cleaning

class ModelBasedAgent:
    def __init__(self):
        # Internal state (memory)
        self.room_state = {
            "Room": "Unknown"
        }
```

```
    def perceive(self, room):
        # Get current perception
        return room

    def update_state(self, room, perception):
        # Update internal state using perception
        self.room_state[room] = perception

    def choose_action(self, room):
        # Decide action using current state and
memory
        if self.room_state[room] == "Dirty":
            return "Clean the room"
        elif self.room_state[room] == "Clean":
            return "Room is already clean"
        else:
            return "Check the room"

    def run(self, room, perception):
        current_perception =
self.perceive(perception)

        # Update internal memory
        self.update_state(room, current_perception)

        # Choose action
        action = self.choose_action(room)

        print("Current Perception :",
current_perception)
        print("Internal State     :",
self.room_state[room])
        print("Action             :", action)


# Create agent
agent = ModelBasedAgent()

# First perception
print("First Observation")
agent.run("Room", "Dirty")

# Second perception
```

```
print("\nSecond Observation")
agent.run("Room", "Clean")
```

Sample Output

```
First Observation
Current Perception : Dirty
Internal State     : Dirty
Action             : Clean the room

Second Observation
Current Perception : Clean
Internal State     : Clean
Action             : Room is already clean
```

Slip 2

Q1).A.Write a python program to implement Tower of Hanoi using recursion. [10 marks]

Program

```
# Tower of Hanoi using Recursion

def tower_of_hanoi(n, source, auxiliary,
destination):

    if n == 1:
        print("Move disk 1 from", source, "to",
destination)
        return

    tower_of_hanoi(n - 1, source, destination,
auxiliary)
```

```
    print("Move disk", n, "from", source, "to",
destination)

    tower_of_hanoi(n - 1, auxiliary, source,
destination)


# Main program
n = int(input("Enter number of disks: "))

tower_of_hanoi(n, 'A', 'B', 'C')
```

Sample Output

```
Enter number of disks: 3

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Q2) B.Write a python program to implement Mini-Max Algorithm.
[20 marks]

Program

```
# Mini-Max Algorithm

def minimax(depth, node_index, is_maximizing, values,
height):

    # If leaf node is reached
    if depth == height:
        return values[node_index]

    if is_maximizing:
        return max(
            minimax(depth + 1, node_index * 2,
                    False, values, height),
            minimax(depth + 1, node_index * 2 + 1,
```

```
                    False, values, height)
        )

    else:
        return min(
            minimax(depth + 1, node_index * 2,
                    True, values, height),
            minimax(depth + 1, node_index * 2 + 1,
                    True, values, height)
        )


# Main Program
values = [3, 5, 2, 9, 12, 5, 23, 23]

height = 3

result = minimax(0, 0, True, values, height)

print("Leaf Node Values:", values)
print("Optimal Value:", result)
```

Sample Output

```
Leaf Node Values: [3, 5, 2, 9, 12, 5, 23, 23]
Optimal Value: 12
```

OR

Q2) B.Write a python program that predicts temperature based on the number of icecreams sold using Linear Regression Create Dataframe which contain temperature and number of icecream sold.

[20 marks]

Program

```
# Linear Regression: Ice Cream Sales and Temperature

import pandas as pd
from sklearn.linear_model import LinearRegression
```

```
# Create DataFrame
data = {
    'Temperature': [20, 22, 25, 28, 30, 32, 35, 38],
    'IceCream_Sold': [100, 120, 150, 180, 210, 240,
280, 320]
}

df = pd.DataFrame(data)

print("Ice Cream Sales Data:")
print(df)

# Independent variable (X)
X = df[['IceCream_Sold']]

# Dependent variable (Y)
Y = df['Temperature']

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X, Y)

# Take input from user
sales = float(input("\nEnter number of ice creams
sold: "))

# Predict temperature
prediction = model.predict([[sales]])

print("Predicted Temperature:", prediction[0])
```

Sample Output

```
Ice Cream Sales Data:
   Temperature  IceCream_Sold
0           20            100
1           22            120
2           25            150
3           28            180
4           30            210
5           32            240
6           35            280
```

```
7           38            320

Enter number of ice creams sold: 200
Predicted Temperature: 29.6
```

Slip 3

Q1)A.Write a python program to simulate a simple reflex agent for fan control. [10 marks]

Program

```
# Simple Reflex Agent for Fan Control

temperature = int(input("Enter room temperature: "))

if temperature >= 30:
    action = "Fan ON"
elif temperature >= 20:
    action = "Fan at LOW speed"
else:
    action = "Fan OFF"

print("Temperature:", temperature, "°C")
print("Agent Action:", action)
```

Sample Output 1

```
Enter room temperature: 35
Temperature: 35 °C
Agent Action: Fan ON
```

Sample Output 2

```
Enter room temperature: 25
Temperature: 25 °C
Agent Action: Fan at LOW speed
```

Q2)B.Write a python program to develop fitness predictors using Random Forest Classification to predict whether a person is FIT or UNFIT based on: Age,

Exercise Hours per Week Accept the input from user and predict for multiple people [20 marks]

Program

```
# Fitness Predictor using Random Forest
Classification

import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# Create training dataset
data = {
    'Age': [18, 20, 22, 25, 28, 30, 35, 40, 45, 50],
    'Exercise_Hours': [6, 7, 5, 6, 4, 5, 3, 2, 2, 1],
    'Fitness': ['FIT', 'FIT', 'FIT', 'FIT', 'FIT',
                'FIT', 'UNFIT', 'UNFIT', 'UNFIT',
'UNFIT']
}

df = pd.DataFrame(data)

print("Training Data:")
print(df)

# Input features
X = df[['Age', 'Exercise_Hours']]

# Target variable
Y = df['Fitness']

# Create Random Forest Classifier
model = RandomForestClassifier(n_estimators=100,
random_state=42)

# Train the model
model.fit(X, Y)

# Accept number of people
n = int(input("\nEnter number of people: "))

# Accept data for multiple people
people = []
```

```
for i in range(n):
    print("\nPerson", i + 1)

    age = int(input("Enter Age: "))
    exercise = float(input("Enter Exercise Hours per
Week: "))

    people.append([age, exercise])

# Create DataFrame for input data
input_df = pd.DataFrame(
    people,
    columns=['Age', 'Exercise_Hours']
)

# Predict fitness
predictions = model.predict(input_df)

# Display results
print("\nFitness Prediction:")
print("-----------------------------")

for i in range(n):
    print(
        "Person", i + 1,
        "Age:", input_df.iloc[i]['Age'],
        "Exercise Hours:",
input_df.iloc[i]['Exercise_Hours'],
        "Fitness:", predictions[i]
    )
```

Sample Output

```
Training Data:
   Age  Exercise_Hours Fitness
0   18               6     FIT
1   20               7     FIT
2   22               5     FIT
3   25               6     FIT
4   28               4     FIT
5   30               5     FIT
6   35               3   UNFIT
7   40               2   UNFIT
```

```
8   45              2   UNFIT
9   50              1   UNFIT

Enter number of people: 3

Person 1
Enter Age: 21
Enter Exercise Hours per Week: 6

Person 2
Enter Age: 38
Enter Exercise Hours per Week: 2

Person 3
Enter Age: 27
Enter Exercise Hours per Week: 5

Fitness Prediction:
-----------------------------
Person 1 Age: 21.0 Exercise Hours: 6.0 Fitness: FIT
Person 2 Age: 38.0 Exercise Hours: 2.0 Fitness: UNFIT
Person 3 Age: 27.0 Exercise Hours: 5.0 Fitness: FIT
```

OR

Q2)B.Write a python program to simulate n-Queens problem. [20 marks]

Program

```
# N-Queens Problem using Backtracking

def is_safe(board, row, col, n):

    # Check the same column
    for i in range(row):
        if board[i] == col:
            return False

    # Check upper-left diagonal
    for i in range(row):
        if board[i] - i == col - row:
            return False
```

```
    # Check upper-right diagonal
    for i in range(row):
        if board[i] + i == col + row:
            return False

    return True


def solve_n_queens(board, row, n):

    # All queens are placed
    if row == n:
        return True

    # Try every column
    for col in range(n):

        if is_safe(board, row, col, n):
            board[row] = col

            # Place queen in next row
            if solve_n_queens(board, row + 1, n):
                return True

            # Backtrack
            board[row] = -1

    return False


# Main program
n = int(input("Enter the value of N: "))

board = [-1] * n

if solve_n_queens(board, 0, n):

    print("\nSolution:")

    for row in range(n):
        for col in range(n):

            if board[row] == col:
```

```
                print("Q", end=" ")
            else:
                print(".", end=" ")

        print()

else:
    print("No solution exists.")
```

Sample Output

```
Enter the value of N: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .
```

Slip 4

Q1)A.Write a python program to predict if a Person is Eligible to Vote (Age ≥ 18) using simple Logistic Classification. Accept person age from user.
[10 marks]

Program

```
# Voting Eligibility using Logistic Regression

import pandas as pd
from sklearn.linear_model import LogisticRegression

# Create training data
data = {
    'Age': [10, 12, 15, 16, 17, 18, 19, 21, 25, 30,
40, 50],
    'Eligible': [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)
```

```
print("Training Data:")
print(df)

# Input feature
X = df[['Age']]

# Target variable
y = df['Eligible']

# Create Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(X, y)

# Accept age from user
age = int(input("\nEnter person's age: "))

# Predict eligibility
prediction = model.predict([[age]])

# Display result
if prediction[0] == 1:
    print("Person is Eligible to Vote.")
else:
    print("Person is Not Eligible to Vote.")
```

Sample Output 1

```
Training Data:
    Age  Eligible
0    10         0
1    12         0
2    15         0
3    16         0
4    17         0
5    18         1
6    19         1
7    21         1
8    25         1
9    30         1
10   40         1
11   50         1
```

```
Enter person's age: 25
Person is Eligible to Vote.
```

Sample Output 2

```
Enter person's age: 16
Person is Not Eligible to Vote.
```

Q2) B.Write a python program to Implement Alpha-Beta Pruning .
[20 marks]

Program

```
# Alpha-Beta Pruning Algorithm

def alpha_beta(depth, node, alpha, beta,
maximizing_player):

    # Leaf node
    if depth == 3:
        return values[node]

    if maximizing_player:
        best = -999

        for i in range(2):
            value = alpha_beta(
                depth + 1,
                node * 2 + i,
                alpha,
                beta,
                False
            )

            best = max(best, value)
            alpha = max(alpha, best)

            # Beta cut-off
            if beta <= alpha:
                print("Beta Pruning at node:", node)
                break
```

```
        return best

    else:
        best = 999

        for i in range(2):
            value = alpha_beta(
                depth + 1,
                node * 2 + i,
                alpha,
                beta,
                True
            )

            best = min(best, value)
            beta = min(beta, best)

            # Alpha cut-off
            if beta <= alpha:
                print("Alpha Pruning at node:", node)
                break

        return best


# Leaf node values
values = [3, 5, 6, 9, 1, 2, 0, -1]

# Initial values
alpha = -999
beta = 999

# Call Alpha-Beta algorithm
result = alpha_beta(0, 0, alpha, beta, True)

print("\nLeaf Node Values:", values)
print("Optimal Value:", result)
```

Sample Output

```
Beta Pruning at node: 2

Leaf Node Values: [3, 5, 6, 9, 1, 2, 0, -1]
Optimal Value: 5
```

OR

Q2) B.Write a Python program to Plot a scatter plot of sepal length vs sepal width for the Iris dataset. Label the axes and add a title give appropriate legend. [20 marks]

Program

```
# Scatter Plot of Iris Dataset

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

# Add target column
df['Species'] = iris.target

# Plot different species
for species in df['Species'].unique():

    data = df[df['Species'] == species]

    plt.scatter(
        data['sepal length (cm)'],
        data['sepal width (cm)'],
        label=iris.target_names[species]
    )

# Add labels and title
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.title("Sepal Length vs Sepal Width - Iris
Dataset")
```

```
# Add legend
plt.legend()

# Display grid and plot
plt.grid(True)
plt.show()
```

Output

The program displays a **scatter plot** with:

- **X-axis:** Sepal Length (cm)
- **Y-axis:** Sepal Width (cm)
- **Title:** Sepal Length vs Sepal Width – Iris Dataset
- **Legend:** Setosa, Versicolor, Virginica

Slip 5

Q1)A.Write a python program to simulate the Monkey Banana Problem. Initialize the monkey's position, perform the necessary actions to reach the banana, and use conditional statements to determine whether the goal state has been achieved. [10 marks]

Program

```
# Monkey Banana Problem

# Initial positions
monkey_position = "door"
box_position = "window"
banana_position = "ceiling"

print("Initial State:")
print("Monkey Position :", monkey_position)
print("Box Position    :", box_position)
print("Banana Position :", banana_position)

# Step 1: Monkey moves to the box
if monkey_position != box_position:
```

```
    print("\nMonkey moves from", monkey_position,
"to", box_position)
    monkey_position = box_position

# Step 2: Monkey pushes the box under the banana
if monkey_position == box_position:
    print("Monkey pushes the box under the banana")
    box_position = banana_position

# Step 3: Monkey climbs the box
if monkey_position == box_position:
    print("Monkey climbs on the box")
    monkey_position = "on_box"

# Step 4: Monkey takes the banana
if monkey_position == "on_box" and box_position ==
banana_position:
    print("Monkey takes the banana")
    banana_obtained = True
else:
    banana_obtained = False

# Check goal state
if banana_obtained:
    print("\nGoal State Achieved!")
    print("Monkey has obtained the banana.")
else:
    print("\nGoal State Not Achieved.")
```

Sample Output

```
Initial State:
Monkey Position : door
Box Position    : window
Banana Position : ceiling

Monkey moves from door to window
Monkey pushes the box under the banana
Monkey climbs on the box
Monkey takes the banana

Goal State Achieved!
Monkey has obtained the banana.
```

Q2)B.Apply Linear Regression on a YouTube Video dataset to predict video views using relevant video attributes. Visualize the relationship between the predictor and target variable using a scatter plot and regression line.
[20 marks]

Program

```
# Linear Regression on YouTube Video Dataset

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Create YouTube dataset
data = {
    'Likes': [100, 250, 400, 550, 700, 900, 1100,
1300, 1500, 1800],
    'Views': [2000, 4500, 7000, 9500, 12000, 15000,
19000, 22000, 25000, 30000]
}

df = pd.DataFrame(data)

print("YouTube Video Dataset:")
print(df)

# Predictor variable
X = df[['Likes']]

# Target variable
Y = df['Views']

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X, Y)

# Predict views
likes = float(input("\nEnter number of likes: "))

prediction = model.predict([[likes]])
```

```
print("Predicted Views:", prediction[0])

# Plot scatter plot
plt.scatter(df['Likes'], df['Views'], label="Actual
Data")

# Regression line
plt.plot(
    df['Likes'],
    model.predict(X),
    label="Regression Line"
)

# Labels and title
plt.xlabel("Number of Likes")
plt.ylabel("Number of Views")
plt.title("YouTube Likes vs Views")

# Legend
plt.legend()

# Display graph
plt.show()
```

Sample Output

```
YouTube Video Dataset:
   Likes  Views
0    100   2000
1    250   4500
2    400   7000
3    550   9500
4    700  12000
5    900  15000
6   1100  19000
7   1300  22000
8   1500  25000
9   1800  30000

Enter number of likes: 1000
Predicted Views: 16962.5
```

OR

Q2) B.Write a pyhton program to create a simple chatbot using predefined rules to respond to user inputs.
[20 marks]

Program

```
print("Simple Chatbot")
print("Type 'bye' to exit the chatbot.")

while True:
    user_input = input("You: ").lower()

    if user_input == "hello" or user_input == "hi":
        print("Bot: Hello! How can I help you?")

    elif user_input == "how are you":
        print("Bot: I am fine. Thank you!")

    elif user_input == "what is your name":
        print("Bot: My name is SimpleBot.")

    elif user_input == "what can you do":
        print("Bot: I can answer simple predefined
questions.")

    elif user_input == "thank you" or user_input ==
"thanks":
        print("Bot: You are welcome!")

    elif user_input == "bye":
        print("Bot: Goodbye! Have a nice day.")
        break

    else:
        print("Bot: Sorry, I don't understand that.")
```

Sample Output

```
Simple Chatbot
Type 'bye' to exit the chatbot.

You: hello
Bot: Hello! How can I help you?
```

```
You: what is your name
Bot: My name is SimpleBot.

You: what can you do
Bot: I can answer simple predefined questions.

You: thanks
Bot: You are welcome!

You: bye
Bot: Goodbye! Have a nice day.
```

Slip 6

Q1)A.Write a python program to implement Hill Climbing Algorithm .
[10 marks]

Program

```
def hill_climbing(values, start):
    current = start

    while True:
        print("Current Position:", current, "Value:",
values[current])

        # Check left and right neighbours
        neighbours = []

        if current > 0:
            neighbours.append(current - 1)

        if current < len(values) - 1:
            neighbours.append(current + 1)

        # Find the best neighbouring position
        best = current

        for position in neighbours:
```

```
            if values[position] > values[best]:
                best = position

        # Stop if no better neighbour is found
        if best == current:
            break

        current = best

    print("Local Maximum:", values[current])
    print("Position:", current)


# Input values
values = [2, 5, 8, 12, 15, 11, 7, 4]

print("Hill Climbing Algorithm")
print("Values:", values)

start = int(input("Enter starting position (0-7): "))

hill_climbing(values, start)
```

Sample Output

```
Hill Climbing Algorithm
Values: [2, 5, 8, 12, 15, 11, 7, 4]

Enter starting position (0-7): 0

Current Position: 0 Value: 2
Current Position: 1 Value: 5
Current Position: 2 Value: 8
Current Position: 3 Value: 12
Current Position: 4 Value: 15
Local Maximum: 15
Position: 4
```

Q2) B.Write a python program to plot a scatter plot of petal length vs petal width for the Iris dataset. Label the axes and add a title.
[20 marks]

Program

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data,
columns=iris.feature_names)

# Plot scatter plot
plt.scatter(
    df['petal length (cm)'],
    df['petal width (cm)']
)

# Add labels and title
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Petal Length vs Petal Width - Iris
Dataset")

# Display graph
plt.show()
```

Expected Graph

Petal Length vs Petal Width - Iris Dataset

Scatter plot of the Iris dataset using petal length and petal width.

00.150.30.451.31.41.51.61.7

OR

Q2) B.Write a python program to implement the Alpha–Beta pruning with depth limit and heuristic. [20 marks]

Program

```
def alpha_beta(depth, node, alpha, beta,
maximizing_player):
```

```
# Depth limit reached
if depth == 3:
    return heuristic[node]

if maximizing_player:
    best = -999

    for i in range(2):
        value = alpha_beta(
            depth + 1,
            node * 2 + i,
            alpha,
            beta,
            False
        )

        best = max(best, value)
        alpha = max(alpha, best)

        # Beta pruning
        if beta <= alpha:
            print("Beta Pruning at node:", node)
            break

    return best

else:
    best = 999

    for i in range(2):
        value = alpha_beta(
            depth + 1,
            node * 2 + i,
            alpha,
            beta,
            True
        )

        best = min(best, value)
        beta = min(beta, best)

        # Alpha pruning
        if beta <= alpha:
            print("Alpha Pruning at node:", node)
```

```
                break

        return best


# Heuristic values of leaf nodes
heuristic = [3, 5, 6, 9, 1, 2, 0, -1]

alpha = -999
beta = 999

print("Alpha-Beta Pruning")
print("Heuristic Values:", heuristic)

result = alpha_beta(0, 0, alpha, beta, True)

print("Optimal Value:", result)
```

Sample Output

```
Alpha-Beta Pruning
Heuristic Values: [3, 5, 6, 9, 1, 2, 0, -1]
Beta Pruning at node: 2
Optimal Value: 5
```

Slip 7

Q1) A.Write a python program to create a DataFrame containing Employee Name, Department and Salary. Display the employees who satisfy all the following conditions: Department = 'IT', Salary > 50000, and Employee Name contains the letter 'a'. [10 marks]

Program:

```
import pandas as pd

# Create employee DataFrame
data = {
    'Employee Name': ['Rahul', 'Amit', 'Sneha',
'Karan', 'Priya', 'Rohan'],
```

```
    'Department': ['IT', 'HR', 'IT', 'IT', 'Finance',
'IT'],
    'Salary': [60000, 55000, 48000, 75000, 65000,
52000]
}

df = pd.DataFrame(data)

print("Employee Data:")
print(df)

# Apply all conditions
result = df[
    (df['Department'] == 'IT') &
    (df['Salary'] > 50000) &
    (df['Employee Name'].str.contains('a',
case=False))
]

print("\nEmployees satisfying all conditions:")
print(result)
```

Sample Output

```
Employee Data:
  Employee Name Department  Salary
0         Rahul         IT   60000
1          Amit         HR   55000
2         Sneha         IT   48000
3         Karan         IT   75000
4         Priya    Finance   65000
5         Rohan         IT   52000

Employees satisfying all conditions:
  Employee Name Department  Salary
0         Rahul         IT   60000
3         Karan         IT   75000
5         Rohan         IT   52000
```

Q2) B.Write a python program to simulate a Model-Based Agent for Smart Home Security.

A smart home security system monitors the main door. The agent should use both the previous door status and the current door status to decide the appropriate security action. The door can have two states: OPEN / CLOSED

The program should accept the previous and current door status from the user and display the appropriate action. Display an error message if the user enters any value other than OPEN or CLOSED.

Program

```
# Model-Based Agent for Smart Home Security

previous_status = input("Enter previous door status
(OPEN/CLOSED): ").upper()
current_status = input("Enter current door status
(OPEN/CLOSED): ").upper()

# Check for valid input
if previous_status not in ["OPEN", "CLOSED"] or
current_status not in ["OPEN", "CLOSED"]:
    print("Error: Invalid door status. Enter only
OPEN or CLOSED.")

else:
    print("\nPrevious Door Status:", previous_status)
    print("Current Door Status :", current_status)

    # Model-based decision
    if previous_status == "CLOSED" and current_status
== "OPEN":
        action = "Door opened. Trigger security
alert!"

    elif previous_status == "OPEN" and current_status
== "CLOSED":
        action = "Door closed. Security status
normal."

    elif previous_status == "OPEN" and current_status
== "OPEN":
        action = "Door remains open. Continue
monitoring."
```

```
    else:
        action = "Door remains closed. Security
status normal."

    print("Security Action      :", action)
```

Sample Output 1

```
Enter previous door status (OPEN/CLOSED): CLOSED
Enter current door status (OPEN/CLOSED): OPEN

Previous Door Status: CLOSED
Current Door Status : OPEN
Security Action      : Door opened. Trigger security
alert!
```

Sample Output 2 – Invalid Input

```
Enter previous door status (OPEN/CLOSED): CLOSE
Enter current door status (OPEN/CLOSED): OPEN

Error: Invalid door status. Enter only OPEN or
CLOSED.
```

[20 marks]

OR

Q2) B.Write a python program to implement Breadth First Search (BFS) for graph traversal. Display the traversal order starting from a given source vertex.

Program

```
from collections import deque

# Graph represented using adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
```

```
def bfs(graph, start):
    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)

    print("BFS Traversal:", end=" ")

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

# Accept source vertex
source = input("Enter source vertex (A-F): ").upper()

if source in graph:
    bfs(graph, source)
else:
    print("Invalid source vertex.")
```

Sample Output

```
Enter source vertex (A-F): A
BFS Traversal: A B C D E F
```

Slip 8

Q1) A.Write a python program to simulate the Monkey Banana Problem. Initialize the monkey's position, perform the necessary actions to reach the banana, and use conditional statements to determine whether the goal state has been achieved. [10 marks]

Program

```
# Monkey Banana Problem

monkey_position = "Door"
box_position = "Window"
banana_position = "Ceiling"

print("Initial State:")
print("Monkey Position :", monkey_position)
print("Box Position    :", box_position)
print("Banana Position :", banana_position)

# Step 1: Move monkey to the box
if monkey_position != box_position:
    print("\nMonkey moves from", monkey_position,
"to", box_position)
    monkey_position = box_position

# Step 2: Push the box below the banana
if monkey_position == box_position:
    print("Monkey pushes the box below the banana")
    box_position = banana_position

# Step 3: Climb on the box
if monkey_position == box_position:
    print("Monkey climbs on the box")
    monkey_position = "On Box"

# Step 4: Take the banana
if monkey_position == "On Box" and box_position ==
banana_position:
    print("Monkey takes the banana")
    banana_obtained = True
else:
    banana_obtained = False

# Check goal state
if banana_obtained:
    print("\nGoal State Achieved!")
    print("Monkey has obtained the banana.")
else:
    print("\nGoal State Not Achieved.")
```

Sample Output

```
Initial State:
Monkey Position : Door
Box Position    : Window
Banana Position : Ceiling

Monkey moves from Door to Window
Monkey pushes the box below the banana
Monkey climbs on the box
Monkey takes the banana

Goal State Achieved!
Monkey has obtained the banana.
```

Q2)B.Write a python program to implement a Logistic Regression classifier for Diabetes Prediction. Train the model using a sample dataset and predict the diabetes status (Diabetic/Non-Diabetic) for multiple persons based on Fasting Blood Sugar, Post-Meal Blood Sugar, Age, and BMI. [20 marks]

Program

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Sample training dataset
data = {
    'Fasting_Blood_Sugar': [85, 90, 95, 100, 110,
120, 130, 140, 150, 160],
    'Post_Meal_Blood_Sugar': [120, 130, 135, 145,
155, 170, 180, 200, 220, 240],
    'Age': [25, 28, 30, 35, 40, 45, 50, 55, 60, 65],
    'BMI': [21, 23, 24, 25, 27, 28, 30, 31, 33, 35],
    'Diabetes': [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

print("Training Dataset:")
```

```
print(df)

# Separate input and output
X = df[['Fasting_Blood_Sugar',
        'Post_Meal_Blood_Sugar',
        'Age',
        'BMI']]

y = df['Diabetes']

# Create and train Logistic Regression model
model = LogisticRegression()
model.fit(X, y)

# Accept number of persons
n = int(input("\nEnter number of persons: "))

persons = []

# Accept details of multiple persons
for i in range(n):
    print("\nPerson", i + 1)

    fasting = float(input("Enter Fasting Blood Sugar:
"))
    post_meal = float(input("Enter Post-Meal Blood
Sugar: "))
    age = float(input("Enter Age: "))
    bmi = float(input("Enter BMI: "))

    persons.append([fasting, post_meal, age, bmi])

# Create DataFrame for input
input_df = pd.DataFrame(
    persons,
    columns=[
        'Fasting_Blood_Sugar',
        'Post_Meal_Blood_Sugar',
        'Age',
        'BMI'
    ]
)

# Predict diabetes status
```

```
predictions = model.predict(input_df)

# Display results
print("\nPrediction Results:")

for i in range(n):
    if predictions[i] == 1:
        status = "Diabetic"
    else:
        status = "Non-Diabetic"

    print("Person", i + 1, ":", status)
```

Sample Output

```
Training Dataset:
   Fasting_Blood_Sugar  Post_Meal_Blood_Sugar  Age
BMI  Diabetes
0                   85                    120   25
21         0
1                   90                    130   28
23         0
2                   95                    135   30
24         0
3                  100                    145   35
25         0
4                  110                    155   40
27         0
5                  120                    170   45
28         1
6                  130                    180   50
30         1
7                  140                    200   55
31         1
8                  150                    220   60
33         1
9                  160                    240   65
35         1

Enter number of persons: 3

Person 1
Enter Fasting Blood Sugar: 90
Enter Post-Meal Blood Sugar: 130
```

```
Enter Age: 25
Enter BMI: 22

Person 2
Enter Fasting Blood Sugar: 130
Enter Post-Meal Blood Sugar: 190
Enter Age: 50
Enter BMI: 30

Person 3
Enter Fasting Blood Sugar: 105
Enter Post-Meal Blood Sugar: 150
Enter Age: 35
Enter BMI: 25

Prediction Results:
Person 1 : Non-Diabetic
Person 2 : Diabetic
Person 3 : Non-Diabetic
```

OR

Q2) B.Write a python program for DataFrame of employees with Salary and Experience: If Experience < 3 years, increase salary by 10%, else increase by 5%. and display updated DataFrame.

[20 marks]

Program

```
import pandas as pd

# Create employee DataFrame
data = {
    'Employee Name': ['Amit', 'Sneha', 'Rahul',
'Priya', 'Karan'],
    'Salary': [40000, 50000, 60000, 45000, 70000],
    'Experience': [2, 4, 1, 3, 5]
}

df = pd.DataFrame(data)
```

```
print("Original Employee Data:")
print(df)

# Update salary based on experience
df['Updated Salary'] = df.apply(
    lambda row: row['Salary'] * 1.10
    if row['Experience'] < 3
    else row['Salary'] * 1.05,
    axis=1
)

print("\nUpdated Employee Data:")
print(df)
```

Sample Output

```
Original Employee Data:
  Employee Name  Salary  Experience
0          Amit   40000           2
1         Sneha   50000           4
2         Rahul   60000           1
3         Priya   45000           3
4         Karan   70000           5

Updated Employee Data:
  Employee Name  Salary  Experience  Updated Salary
0          Amit   40000           2         44000.0
1         Sneha   50000           4         52500.0
2         Rahul   60000           1         66000.0
3         Priya   45000           3         47250.0
4         Karan   70000           5         73500.0
```

Slip 9

Q1) A.Write a program to create a DataFrame which contains a person name and age ,Sort a DataFrame in ascending order of Age
[10 marks]

Program

```
import pandas as pd

# Create DataFrame
data = {
    'Person Name': ['Rahul', 'Sneha', 'Amit',
'Priya', 'Karan'],
    'Age': [25, 20, 30, 22, 27]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Sort DataFrame by Age in ascending order
sorted_df = df.sort_values(by='Age', ascending=True)

print("\nDataFrame Sorted by Age:")
print(sorted_df)
```

Sample Output

```
Original DataFrame:
  Person Name  Age
0       Rahul   25
1       Sneha   20
2        Amit   30
3       Priya   22
4       Karan   27

DataFrame Sorted by Age:
  Person Name  Age
1       Sneha   20
3       Priya   22
0       Rahul   25
4       Karan   27
2        Amit   30
```

Q2)B.Write a program to implement the Blocks World Problem. Represent the initial and goal states of a set of blocks, perform valid block movements

according to the problem constraints and display each intermediate state until the goal configuration is achieved. [20 marks]

Program

```
# Blocks World Problem

def display_state(state):
    print("State:")
    for block, position in state.items():
        print(block, "->", position)
    print()


# Initial State
initial_state = {
    'A': 'Table',
    'B': 'A',
    'C': 'Table'
}

# Goal State
goal_state = {
    'A': 'Table',
    'B': 'C',
    'C': 'Table'
}

print("Initial State:")
display_state(initial_state)

# Move B from A to Table
print("Move 1: Move B from A to Table")
initial_state['B'] = 'Table'
display_state(initial_state)

# Move B from Table to C
print("Move 2: Move B from Table to C")
initial_state['B'] = 'C'
display_state(initial_state)

# Check goal state
if initial_state == goal_state:
```

```
    print("Goal State Achieved!")
else:
    print("Goal State Not Achieved.")
```

Sample Output

```
Initial State:
State:
A -> Table
B -> A
C -> Table

Move 1: Move B from A to Table
State:
A -> Table
B -> Table
C -> Table

Move 2: Move B from Table to C
State:
A -> Table
B -> C
C -> Table

Goal State Achieved!
```

OR

Q2) B.Write a python program to compare BFS and DFS performance for graph search problems.

[20 marks]

Program

```
from collections import deque
import time

# Graph represented using adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
```

```
    'E': [],
    'F': [],
    'G': ['I'],
    'H': [],
    'I': []
}


# Breadth First Search
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    order = []

    while queue:
        vertex = queue.popleft()

        if vertex not in visited:
            visited.add(vertex)
            order.append(vertex)

            for neighbour in graph[vertex]:
                if neighbour not in visited:
                    queue.append(neighbour)

    return order


# Depth First Search
def dfs(graph, start):
    visited = set()
    stack = [start]
    order = []

    while stack:
        vertex = stack.pop()

        if vertex not in visited:
            visited.add(vertex)
            order.append(vertex)

            # Reverse order to maintain left-to-right
traversal
            for neighbour in reversed(graph[vertex]):
```

```
                if neighbour not in visited:
                    stack.append(neighbour)

    return order


source = input("Enter source vertex (A-I): ").upper()

if source not in graph:
    print("Invalid source vertex.")
else:
    # Measure BFS performance
    start_time = time.perf_counter()
    bfs_result = bfs(graph, source)
    bfs_time = time.perf_counter() - start_time

    # Measure DFS performance
    start_time = time.perf_counter()
    dfs_result = dfs(graph, source)
    dfs_time = time.perf_counter() - start_time

    print("\nBFS Traversal:", " ->
".join(bfs_result))
    print("BFS Time:", format(bfs_time, ".8f"),
"seconds")

    print("\nDFS Traversal:", " ->
".join(dfs_result))
    print("DFS Time:", format(dfs_time, ".8f"),
"seconds")

    print("\nPerformance Comparison")
    print("----------------------")
    print("BFS Visited Nodes:", len(bfs_result))
    print("DFS Visited Nodes:", len(dfs_result))
```

Sample Output

```
Enter source vertex (A-I): A

BFS Traversal: A -> B -> C -> D -> E -> F -> G -> H -
> I
BFS Time: 0.00000520 seconds
```

```
DFS Traversal: A -> B -> D -> H -> E -> C -> F -> G -
> I
DFS Time: 0.00000410 seconds

Performance Comparison
----------------------
BFS Visited Nodes: 9
DFS Visited Nodes: 9
```

Slip 11

Q1) A.Write a python program to read the salary_data.csv dataset, train a Linear Regression model using Years of Experience as the feature and Salary as the target and predict the salary for a new employee.
[10 marks]

Program

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Read the CSV dataset
df = pd.read_csv("salary_data.csv")

print("Salary Dataset:")
print(df)

# Select feature and target
X = df[['YearsExperience']]
y = df['Salary']

# Create and train Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Accept experience of new employee
experience = float(input("\nEnter Years of
Experience: "))
```

```
# Create DataFrame for prediction
new_employee = pd.DataFrame(
    [[experience]],
    columns=['YearsExperience']
)

# Predict salary
predicted_salary = model.predict(new_employee)

print("Predicted Salary:", predicted_salary[0])
```

Sample `salary_data.csv`

The CSV file should be in the same folder as the Python program.

```
YearsExperience,Salary
1.1,39343
1.3,46205
1.5,37731
2.0,43525
2.2,39891
3.0,56642
3.2,54445
3.7,57189
4.0,56957
4.5,61111
5.0,67938
6.0,93940
7.0,98273
8.0,101302
9.0,105582
10.0,121872
```

Sample Output

```
Salary Dataset:
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
...
15             10.0  121872
```

```
Enter Years of Experience: 6.5
Predicted Salary: 83900.XX
```

Q2) B.Write a python program to implement the Water Jug Problem.
[20 marks]

Program

```
from collections import deque

def water_jug_problem(capacity1, capacity2, target):
    visited = set()
    queue = deque()

    # Initial state
    queue.append(((0, 0), []))
    visited.add((0, 0))

    while queue:
        (jug1, jug2), path = queue.popleft()

        # Goal test
        if jug1 == target or jug2 == target:
            print("\nSolution Path:")
            for step in path:
                print(step)

            print("Goal State:", (jug1, jug2))
            return

        # Generate possible states
        states = []

        # Fill Jug 1
        states.append(((capacity1, jug2), "Fill Jug
1"))

        # Fill Jug 2
        states.append(((jug1, capacity2), "Fill Jug
2"))

        # Empty Jug 1
        states.append(((0, jug2), "Empty Jug 1"))
```

```
        # Empty Jug 2
        states.append(((jug1, 0), "Empty Jug 2"))

        # Pour Jug 1 into Jug 2
        amount = min(jug1, capacity2 - jug2)
        states.append(
            ((jug1 - amount, jug2 + amount),
             "Pour Jug 1 -> Jug 2")
        )

        # Pour Jug 2 into Jug 1
        amount = min(jug2, capacity1 - jug1)
        states.append(
            ((jug1 + amount, jug2 - amount),
             "Pour Jug 2 -> Jug 1")
        )

        # Add unvisited states to queue
        for new_state, action in states:
            if new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path +
[action]))

    print("No solution exists.")


# Jug capacities and target
capacity1 = 4
capacity2 = 3
target = 2

print("WATER JUG PROBLEM")
print("-----------------")
print("Jug 1 Capacity:", capacity1, "litres")
print("Jug 2 Capacity:", capacity2, "litres")
print("Target:", target, "litres")

water_jug_problem(capacity1, capacity2, target)
```

Sample Output

```
WATER JUG PROBLEM
```

```
-----------------
Jug 1 Capacity: 4 litres
Jug 2 Capacity: 3 litres
Target: 2 litres

Solution Path:
Fill Jug 2
Pour Jug 2 -> Jug 1
Empty Jug 1
Pour Jug 2 -> Jug 1
Fill Jug 2
Pour Jug 2 -> Jug 1
Goal State: (4, 2)
```

OR

Q2) B.Write a python program using Logistic Regression to detect credit card fraud using the following features: Transaction Amount, Card Age (in years), Transaction Time (023),Location Match Score (0–100),Online Transaction (1 = Yes, 0 = No),Train the model and predict fraud (1) or legit (0) based on user input. [20 marks]

Program

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Training dataset
data = {
    'Transaction_Amount': [100, 500, 50, 2000, 1500,
80, 3000, 120, 2500, 70],
    'Card_Age': [5, 4, 8, 1, 2, 10, 1, 7, 2, 9],
    'Transaction_Time': [10, 14, 9, 23, 22, 11, 2,
15, 1, 13],
    'Location_Match_Score': [95, 90, 98, 20, 30, 97,
10, 92, 25, 96],
    'Online_Transaction': [0, 1, 0, 1, 1, 0, 1, 0, 1,
0],
    'Fraud': [0, 0, 0, 1, 1, 0, 1, 0, 1, 0]
}
```

```
df = pd.DataFrame(data)

print("Training Dataset:")
print(df)

# Features
X = df[['Transaction_Amount',
        'Card_Age',
        'Transaction_Time',
        'Location_Match_Score',
        'Online_Transaction']]

# Target
y = df['Fraud']

# Create and train Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X, y)

# Accept user input
print("\nEnter Transaction Details")

amount = float(input("Enter Transaction Amount: "))
card_age = float(input("Enter Card Age (in years):
"))
transaction_time = float(input("Enter Transaction
Time (0-23): "))
location_score = float(input("Enter Location Match
Score (0-100): "))
online = int(input("Online Transaction (1=Yes, 0=No):
"))

# Create DataFrame for prediction
new_transaction = pd.DataFrame(
    [[amount, card_age, transaction_time,
      location_score, online]],
    columns=[
        'Transaction_Amount',
        'Card_Age',
        'Transaction_Time',
        'Location_Match_Score',
        'Online_Transaction'
    ]
)
```

```
# Predict transaction
prediction = model.predict(new_transaction)

# Display result
if prediction[0] == 1:
    print("\nPrediction: FRAUD (1)")
else:
    print("\nPrediction: LEGIT (0)")
```

Sample Output

```
Training Dataset:
T_Amount Card_Age  T_Time Location_Match_Score
Online_T  Fraud
 100         5      10        95                    0
0
 500         4      14        90                    1
0
...

Enter Transaction Details
Enter Transaction Amount: 2000
Enter Card Age (in years): 1
Enter Transaction Time (0-23): 23
Enter Location Match Score (0-100): 20
Online Transaction (1=Yes, 0=No): 1

Prediction: FRAUD (1)
```

Slip 12

Q1)A.Write a python program to read the car_data.csv dataset and visualize the relationship between Car_Age and Selling_Price using a Scatter Plot.
[10 marks]

Program

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

# Read the CSV dataset
df = pd.read_csv("car_data.csv")

# Display the dataset
print("Car Dataset:")
print(df)

# Create scatter plot
plt.scatter(df['Car_Age'], df['Selling_Price'])

# Add labels and title
plt.xlabel("Car Age")
plt.ylabel("Selling Price")
plt.title("Car Age vs Selling Price")

# Display grid
plt.grid(True)

# Display the graph
plt.show()
```

Sample Output

```
Car Dataset:
   Car_Age  Selling_Price
0        1         480000
1        2         450000
2        3         400000
3        4         350000
4        5         300000
...
```

Q2)B.Write a python program to implement Tower of Hanoi using recursion. [20 marks]

Program

```
# Function to solve Tower of Hanoi
def tower_of_hanoi(n, source, auxiliary,
destination):
```

```
    # Base condition
    if n == 1:
        print("Move disk 1 from", source, "to",
destination)
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, destination,
auxiliary)

    # Move the largest disk from source to
destination
    print("Move disk", n, "from", source, "to",
destination)

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi(n - 1, auxiliary, source,
destination)


# Accept number of disks
n = int(input("Enter number of disks: "))

# Call the function
tower_of_hanoi(n, 'A', 'B', 'C')
```

Sample Output

For `n = 3`:

```
Enter number of disks: 3

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

OR

Q2) B.Write a python program to simulate a Vacuum Cleaner Agent. The agent should accept the current location and the cleanliness status of the room as input and decide whether to move to another location, clean the room, or remain in the current location. Display the agent's action based on the given conditions.

Program

```
# Vacuum Cleaner Agent

location = input("Enter current location (A/B):
").upper()
status = input("Enter room status (CLEAN/DIRTY):
").upper()

if location not in ["A", "B"]:
    print("Invalid location.")
elif status not in ["CLEAN", "DIRTY"]:
    print("Invalid cleanliness status.")
else:
    print("\nCurrent Location :", location)
    print("Room Status      :", status)

    if status == "DIRTY":
        action = "Clean the room"

    elif location == "A" and status == "CLEAN":
        action = "Move to location B"

    elif location == "B" and status == "CLEAN":
        action = "Move to location A"

    print("Agent Action     :", action)
```

Sample Output 1

```
Enter current location (A/B): A
Enter room status (CLEAN/DIRTY): DIRTY

Current Location : A
Room Status      : DIRTY
Agent Action     : Clean the room
```

Sample Output 2

```
Enter current location (A/B): A
Enter room status (CLEAN/DIRTY): CLEAN

Current Location : A
Room Status      : CLEAN
Agent Action     : Move to location B
```

Slip 17

Q1)A.Write a Python Program to Split the Iris dataset into training and testing sets (80-20) and print the shapes of X_train and X_test.
[10 marks]

Program

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data,
columns=iris.feature_names)

# Features and target
X = df
y = iris.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Print shapes
print("Shape of X_train:", X_train.shape)
print("Shape of X_test :", X_test.shape)
```

Sample Output

```
Shape of X_train: (120, 4)
Shape of X_test : (30, 4)
```

Q2)B.Write a python program to implement a Model-Based Agent for Smart Traffic Signal Control by maintaining previous and current traffic density before deciding the signal status. [20 marks]

Program

```
# Model-Based Agent for Smart Traffic Signal Control

previous_density = input(
    "Enter previous traffic density
(LOW/MEDIUM/HIGH): "
).upper()

current_density = input(
    "Enter current traffic density (LOW/MEDIUM/HIGH):
"
).upper()

# Validate input
valid_density = ["LOW", "MEDIUM", "HIGH"]

if previous_density not in valid_density or
current_density not in valid_density:
    print("Error: Enter density as LOW, MEDIUM or
HIGH.")

else:
    # Display current model state
    print("\nPrevious Traffic Density :",
previous_density)
    print("Current Traffic Density  :",
current_density)

    # Decide traffic signal
    if current_density == "HIGH":
        signal = "GREEN - Allow traffic to move"

    elif current_density == "MEDIUM":
```

```
        signal = "GREEN for limited time"

    else:
        signal = "RED - Stop traffic"

    # Consider change in traffic density
    if previous_density != current_density:
        print("Traffic Density Changed  : Yes")
    else:
        print("Traffic Density Changed  : No")

    print("Signal Status             :", signal)
```

Sample Output 1

```
Enter previous traffic density (LOW/MEDIUM/HIGH):
MEDIUM
Enter current traffic density (LOW/MEDIUM/HIGH): HIGH

Previous Traffic Density : MEDIUM
Current Traffic Density  : HIGH
Traffic Density Changed  : Yes
Signal Status             : GREEN - Allow traffic to
move
```

Sample Output 2

```
Enter previous traffic density (LOW/MEDIUM/HIGH):
HIGH
Enter current traffic density (LOW/MEDIUM/HIGH): LOW

Previous Traffic Density : HIGH
Current Traffic Density  : LOW
Traffic Density Changed  : Yes
Signal Status             : RED - Stop traffic
```

OR

Q2)B.Write a python program to perform Linear Regression using a suitable dataset and predict the output for user-defined input values.
[20 marks]

Program

Here, **Hours_Studied** is used as the input feature and **Marks** as the target.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Create a suitable dataset
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks': [35, 40, 45, 50, 55, 60, 65, 70, 80, 85]
}

df = pd.DataFrame(data)

print("Student Dataset:")
print(df)

# Select feature and target
X = df[['Hours_Studied']]
y = df['Marks']

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Accept user input
hours = float(input("\nEnter number of hours studied:
"))

# Create DataFrame for prediction
new_data = pd.DataFrame(
    [[hours]],
    columns=['Hours_Studied']
)

# Predict marks
prediction = model.predict(new_data)

print("Predicted Marks:", prediction[0])
```

Sample Output

```
Student Dataset:
```

```
   Hours_Studied  Marks
0              1     35
1              2     40
2              3     45
3              4     50
4              5     55
5              6     60
6              7     65
7              8     70
8              9     80
9             10     85

Enter number of hours studied: 7.5
Predicted Marks: 69.75
```

Slip 18

Q1)A.Write a python program to check whether a given string is a Palindrome. [10 marks]

Program

```
# Program to check whether a string is a palindrome

string = input("Enter a string: ")

# Reverse the string
reverse_string = string[::-1]

# Check palindrome
if string == reverse_string:
    print("The given string is a Palindrome.")
else:
    print("The given string is not a Palindrome.")
```

Sample Output 1

```
Enter a string: madam
The given string is a Palindrome.
```

Sample Output 2

```
Enter a string: hello
The given string is not a Palindrome.
```

Q2) B.Write a python program to implement DataFrame operations on employee records, including sorting, filtering and updating records using the Pandas library. [20 marks]

Program

```
import pandas as pd

# Create employee dataset
data = {
    'Employee_ID': [101, 102, 103, 104, 105],
    'Employee_Name': ['Amit', 'Sneha', 'Rahul',
'Priya', 'Karan'],
    'Department': ['IT', 'HR', 'IT', 'Finance',
'IT'],
    'Salary': [50000, 45000, 60000, 55000, 48000],
    'Experience': [2, 3, 5, 4, 1]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Original Employee Data:")
print(df)

# 1. Sorting employees by Salary
sorted_df = df.sort_values(by='Salary',
ascending=False)

print("\nEmployees Sorted by Salary:")
print(sorted_df)

# 2. Filtering employees from IT department
filtered_df = df[df['Department'] == 'IT']

print("\nEmployees from IT Department:")
print(filtered_df)
```

```
# 3. Updating salary of employees with experience
less than 3 years
df.loc[df['Experience'] < 3, 'Salary'] = (
    df.loc[df['Experience'] < 3, 'Salary'] * 1.10
)

print("\nEmployee Data After Salary Update:")
print(df)
```

Sample Output

```
Original Employee Data:
   Employee_ID Employee_Name Department  Salary
Experience
0          101          Amit         IT   50000
2
1          102         Sneha         HR   45000
3
2          103         Rahul         IT   60000
5
3          104         Priya    Finance   55000
4
4          105         Karan         IT   48000
1

Employees Sorted by Salary:
   Employee_ID Employee_Name Department  Salary
Experience
2          103         Rahul         IT   60000
5
3          104         Priya    Finance   55000
4
0          101          Amit         IT   50000
2
4          105         Karan         IT   48000
1
1          102         Sneha         HR   45000
3

Employees from IT Department:
   Employee_ID Employee_Name Department  Salary
Experience
0          101          Amit         IT   50000
2
```

```
2          103         Rahul          IT   60000
5
4          105         Karan          IT   48000
1

Employee Data After Salary Update:
   Employee_ID Employee_Name Department  Salary
Experience
0          101          Amit          IT   55000
2
1          102         Sneha          HR   45000
3
2          103         Rahul          IT   60000
5
3          104         Priya     Finance   55000
4
4          105         Karan          IT   52800
1
```

OR

Q2) B.Develop a python program to demonstrate the Perception → Decision → Action cycle by simulating an Intelligent Agent in a simple environment.
[20 marks]

Program

```
# Intelligent Agent: Perception -> Decision -> Action

# Simple environment
environment = ["Dirty", "Clean", "Dirty", "Clean",
"Dirty"]

print("Initial Environment:")
print(environment)

print("\nAgent Working:")
print("----------------")

# Agent moves through each location
for i in range(len(environment)):

    # 1. Perception
```

```
    perception = environment[i]
    print("\nLocation:", i + 1)
    print("Perception:", perception)

    # 2. Decision
    if perception == "Dirty":
        decision = "Clean the location"
    else:
        decision = "Move to next location"

    print("Decision:", decision)

    # 3. Action
    if perception == "Dirty":
        environment[i] = "Clean"
        action = "Location cleaned"
    else:
        action = "No cleaning required"

    print("Action:", action)

print("\nFinal Environment:")
print(environment)
```

Sample Output

```
Initial Environment:
['Dirty', 'Clean', 'Dirty', 'Clean', 'Dirty']

Agent Working:
----------------

Location: 1
Perception: Dirty
Decision: Clean the location
Action: Location cleaned

Location: 2
Perception: Clean
Decision: Move to next location
Action: No cleaning required

Location: 3
Perception: Dirty
```

```
Decision: Clean the location
Action: Location cleaned

Location: 4
Perception: Clean
Decision: Move to next location
Action: No cleaning required

Location: 5
Perception: Dirty
Decision: Clean the location
Action: Location cleaned

Final Environment:
['Clean', 'Clean', 'Clean', 'Clean', 'Clean']
```

Slip 19

Q1)A.Write a program to create a DataFrame which contains a person name and age ,Sort a DataFrame in ascending order of Age
[10 marks]

Program

```
import pandas as pd

# Create data
data = {
    'Person Name': ['Rahul', 'Sneha', 'Amit',
'Priya', 'Karan'],
    'Age': [25, 20, 30, 22, 27]
}

# Create DataFrame
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)
```

```
# Sort DataFrame by Age in ascending order
sorted_df = df.sort_values(by='Age', ascending=True)

print("\nDataFrame Sorted by Age:")
print(sorted_df)
```

Sample Output

```
Original DataFrame:
  Person Name  Age
0       Rahul   25
1       Sneha   20
2        Amit   30
3       Priya   22
4       Karan   27

DataFrame Sorted by Age:
  Person Name  Age
1       Sneha   20
3       Priya   22
0       Rahul   25
4       Karan   27
2        Amit   30
```

Q2)B.Write a python program to load the Diabetic in built dataset, convert it into a Data frames and display the first 7 rows and the shape of the dataset. [20 marks]

Program

```
import pandas as pd
from sklearn.datasets import load_diabetes

# Load the built-in Diabetes dataset
diabetes = load_diabetes()

# Convert dataset into DataFrame
df = pd.DataFrame(
    diabetes.data,
    columns=diabetes.feature_names
)

# Display first 7 rows
```

```
print("First 7 Rows of Diabetes Dataset:")
print(df.head(7))

# Display shape of dataset
print("\nShape of Dataset:")
print(df.shape)
```

OR

Q2)B.Write a python program to using Logistic Regressions to detect credit card fraud using the following features: Transaction Amount ,Card Age(calculated into years), Transaction Time(0-23),Location Matching scores(0-100),Online Transcation (0=NO,1=Yes)Train the model and predict fraud(1) or legit (0) based on user input. [20 marks]

Program

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Create training dataset
data = {
    'Transaction_Amount': [100, 500, 50, 2000, 1500,
80, 3000, 120, 2500, 70],
    'Card_Age': [5, 4, 8, 1, 2, 10, 1, 7, 2, 9],
    'Transaction_Time': [10, 14, 9, 23, 22, 11, 2,
15, 1, 13],
    'Location_Match_Score': [95, 90, 98, 20, 30, 97,
10, 92, 25, 96],
    'Online_Transaction': [0, 1, 0, 1, 1, 0, 1, 0, 1,
0],
    'Fraud': [0, 0, 0, 1, 1, 0, 1, 0, 1, 0]
}

# Convert data into DataFrame
df = pd.DataFrame(data)

print("Training Dataset:")
print(df)

# Select input features
```

```
X = df[
    [
        'Transaction_Amount',
        'Card_Age',
        'Transaction_Time',
        'Location_Match_Score',
        'Online_Transaction'
    ]
]

# Select target
y = df['Fraud']

# Create and train Logistic Regression model
model = LogisticRegression(max_iter=1000)
model.fit(X, y)

# Take user input
print("\nEnter Transaction Details")

amount = float(input("Enter Transaction Amount: "))
card_age = float(input("Enter Card Age (in years):
"))
transaction_time = int(input("Enter Transaction Time
(0-23): "))
location_score = float(input("Enter Location Matching
Score (0-100): "))
online = int(input("Online Transaction (0=No, 1=Yes):
"))

# Create DataFrame for new transaction
new_transaction = pd.DataFrame(
    [[
        amount,
        card_age,
        transaction_time,
        location_score,
        online
    ]],
    columns=[
        'Transaction_Amount',
        'Card_Age',
        'Transaction_Time',
        'Location_Match_Score',
```

```
        'Online_Transaction'
    ]
)

# Predict fraud
prediction = model.predict(new_transaction)

# Display result
if prediction[0] == 1:
    print("\nPrediction: FRAUD (1)")
else:
    print("\nPrediction: LEGIT (0)")
```

Sample Output

```
Training Dataset:
   T_Amount  Card_Age  T_Time  Location_Match_Score
Online_T     Fraud
0     100         5      10              95
0           0
1     500         4      14              90
1           0
2      50         8       9              98
0           0
3     2000        1      23              20
1           1
4     1500        2      22              30
1           1
...

Enter Transaction Details
Enter Transaction Amount: 2000
Enter Card Age (in years): 1
Enter Transaction Time (0-23): 23
Enter Location Matching Score (0-100): 20
Online Transaction (0=No, 1=Yes): 1
Prediction: FRAUD (1)
```

Slip 20

Q1)A.Write a python program for Multiple Linear Regression that loads data from a CSV file to predict Sales based on Advertising Budget (TV, Radio, Newspaper). [10 marks]

Program

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Load data from CSV file
df = pd.read_csv("advertising.csv")

print("Advertising Dataset:")
print(df)

# Select input features
X = df[['TV', 'Radio', 'Newspaper']]

# Select target variable
y = df['Sales']

# Create and train Multiple Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Accept user input
tv = float(input("\nEnter TV Advertising Budget: "))
radio = float(input("Enter Radio Advertising Budget:
"))
newspaper = float(input("Enter Newspaper Advertising
Budget: "))

# Create DataFrame for new input
new_data = pd.DataFrame(
    [[tv, radio, newspaper]],
    columns=['TV', 'Radio', 'Newspaper']
)

# Predict Sales
prediction = model.predict(new_data)
```

```
print("\nPredicted Sales:", prediction[0])
```

Sample `advertising.csv`

Save the CSV file in the **same folder** as the Python program.

```
TV,Radio,Newspaper,Sales
230.1,37.8,69.2,22.1
44.5,39.3,45.1,10.4
17.2,45.9,69.3,9.3
151.5,41.3,58.5,18.5
180.8,10.8,58.4,12.9
8.7,48.9,75.0,7.2
57.5,32.8,23.5,11.8
120.2,19.6,11.6,13.2
199.8,2.6,21.2,10.6
66.1,5.8,24.2,8.6
```

Sample Input

```
Enter TV Advertising Budget: 150
Enter Radio Advertising Budget: 30
Enter Newspaper Advertising Budget: 40
```

Sample Output

```
Predicted Sales: 17.XX
```

Q2) B.Write a python program clean the dataset by handling missing and duplicate values use any Dataset

a. Check missing values

b. Check duplicated rows.

c. Remove duplicates

d. Store cleaned dataset in a new CSV file.
[20 marks]

OR

Q2) B.Write a program to for DataFrame of customer with Salary and Experience: If Experience < 5 years, increase salary by 11.5%, else increase by 6.5%. and display updated DataFrame.

Python program using Pandas DataFrame:

```
import pandas as pd

# Create DataFrame
data = {
    "Customer": ["Amit", "Sneha", "Rahul", "Priya",
"Neha"],
    "Salary": [30000, 40000, 50000, 45000, 60000],
    "Experience": [2, 4, 6, 3, 8]
}

df = pd.DataFrame(data)

# Display original DataFrame
print("Original DataFrame:")
print(df)

# Increase salary based on experience
df.loc[df["Experience"] < 5, "Salary"] *= 1.115
df.loc[df["Experience"] >= 5, "Salary"] *= 1.065

# Display updated DataFrame
print("\nUpdated DataFrame:")
print(df)
```

Output

```
Original DataFrame:
  Customer  Salary  Experience
0     Amit   30000           2
1    Sneha   40000           4
2    Rahul   50000           6
3    Priya   45000           3
4     Neha   60000           8

Updated DataFrame:
  Customer   Salary  Experience
0     Amit  33450.0           2
```

```
1    Sneha  44600.0           4
2    Rahul  53250.0           6
3    Priya  50175.0           3
4     Neha  63900.0           8
```

Slip 21

Q1)A.Write a python program to predict if a student is Eligible to take admission (marks ≥ 55) using simple Logistic Classification. Accept student marks from user. [10 marks]

Program

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Training data
marks = np.array([30, 40, 45, 50, 55, 60, 65, 70, 80,
90]).reshape(-1, 1)

# 0 = Not Eligible, 1 = Eligible
admission = [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]

# Create and train Logistic Regression model
model = LogisticRegression()
model.fit(marks, admission)

# Accept marks from user
student_marks = float(input("Enter student's marks:
"))

# Predict eligibility
prediction = model.predict([[student_marks]])

# Display result
if prediction[0] == 1:
    print("Student is Eligible to take admission.")
```

```
else:
    print("Student is Not Eligible to take
admission.")
```

Sample Output

```
Enter student's marks: 65
Student is Eligible to take admission.
```

Another example:

```
Enter student's marks: 45
Student is Not Eligible to take admission.
```

Q2)B.write a python program to load item.csv containing cost and type of item and perform basic data exploration

a. Display the first four rows of item dataset

b. Print the shape of the dataset.

c. Show column names.

d. Identify missing values

e. Display dataset inf
[20 marks]

Program: Basic Data Exploration of `item.csv`

Assume the `item.csv` file contains two columns: **Cost** and **Type**.

`item.csv`

```
Cost,Type
500,Electronics
250,Stationery
1200,Furniture
300,Clothing
800,Electronics
450,Stationery
```

Program

```
import pandas as pd
```

```
# Load item.csv
df = pd.read_csv("item.csv")

# a. Display the first four rows
print("First four rows of the dataset:")
print(df.head(4))

# b. Print the shape of the dataset
print("\nShape of the dataset:")
print(df.shape)

# c. Show column names
print("\nColumn names:")
print(df.columns)

# d. Identify missing values
print("\nMissing values:")
print(df.isnull().sum())

# e. Display dataset information
print("\nDataset Information:")
df.info()
```

Output

```
First four rows of the dataset:
   Cost         Type
0   500  Electronics
1   250   Stationery
2  1200    Furniture
3   300     Clothing

Shape of the dataset:
(6, 2)

Column names:
Index(['Cost', 'Type'], dtype='object')

Missing values:
Cost    0
Type    0
dtype: int64
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Cost    6 non-null      int64
 1   Type    6 non-null      object
dtypes: int64(1), object(1)
memory usage: 228.0+ bytes
```

OR

Q2)B.Write a python program to simulate a Goal-Based Agent for a simple navigation problem. The agent should accept the current location and goal location as input and determine the next action required to move toward the goal. Display the current state, goal state, and selected action.

[20 marks]

Program:

A → B → C → D → E

```
# Goal-Based Agent for a simple navigation problem

# Define the navigation path
path = ["A", "B", "C", "D", "E"]

# Accept input from user
current_location = input("Enter current location (A-
E): ").upper()
goal_location = input("Enter goal location (A-E):
").upper()

# Display current state and goal state
print("\nCurrent State :", current_location)
print("Goal State    :", goal_location)

# Find positions of current and goal locations
current_position = path.index(current_location)
```

```
goal_position = path.index(goal_location)

# Determine the next action
if current_position < goal_position:
    next_location = path[current_position + 1]
    action = "Move Forward to " + next_location

elif current_position > goal_position:
    next_location = path[current_position - 1]
    action = "Move Backward to " + next_location

else:
    action = "Goal Reached"

# Display selected action
print("Selected Action:", action)
```

Sample Output 1

```
Enter current location (A-E): A
Enter goal location (A-E): D

Current State : A
Goal State    : D
Selected Action: Move Forward to B
```

Sample Output 2

```
Enter current location (A-E): D
Enter goal location (A-E): B

Current State : D
Goal State    : B
Selected Action: Move Backward to C
```

Slip 22

Q1)A.Write a python program to check whether a given number is a Prime Number using a user-defined function.
[10 marks]

Program: Check Whether a Number is Prime Using a User-Defined Function

```
# Function to check whether a number is prime
def check_prime(n):
    if n <= 1:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True


# Accept number from user
num = int(input("Enter a number: "))

# Check and display result
if check_prime(num):
    print(num, "is a Prime Number")
else:
    print(num, "is not a Prime Number")
```

Sample Output

```
Enter a number: 17
17 is a Prime Number
```

Another example:

```
Enter a number: 20
20 is not a Prime Number
```

Q2)B.Write a python program to load employee.csv containing Employee ID, Salary, and Department dataset and perform basic data exploration.

a.Display the first four rows of the dataset. b. Print the shape of the dataset. c. Show column names. d. Identify missing values. e. Display dataset information. [20 marks]

Basic Data Exploration of `employee.csv`

```
import pandas as pd

# Load the employee.csv file
df = pd.read_csv("employee.csv")

# a. Display the first four rows
print("First Four Rows:")
print(df.head(4))

# b. Print the shape of the dataset
print("\nShape of Dataset:")
print(df.shape)

# c. Show column names
print("\nColumn Names:")
print(df.columns)

# d. Identify missing values
print("\nMissing Values:")
print(df.isnull().sum())

# e. Display dataset information
print("\nDataset Information:")
df.info()
```

Explanation

Part	Statement	Purpose
a	`df.head(4)`	Displays the first 4 rows
b	`df.shape`	Shows number of rows and columns
c	`df.columns`	Displays all column names
d	`df.isnull().sum()`	Counts missing values in each column

Part	Statement	Purpose
e	`df.info()`	Displays column names, data types, non-null values, and memory usage

Sample `employee.csv`

```
Employee ID,Salary,Department
101,35000,HR
102,45000,IT
103,40000,Sales
104,55000,IT
105,30000,HR
```

Sample Output

```
First Four Rows:
   Employee ID  Salary Department
0          101   35000         HR
1          102   45000         IT
2          103   40000      Sales
3          104   55000         IT

Shape of Dataset:
(5, 3)

Column Names:
Index(['Employee ID', 'Salary', 'Department'],
dtype='object')

Missing Values:
Employee ID    0
Salary         0
Department     0
dtype: int64

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 Employee ID    5 non-null int64
 Salary         5 non-null int64
 Department     5 non-null object
```

```
dtypes: int64(2), object(1)
```

OR

Q2)B.Write a python program that predicts the price of a house using a dataset that includes Area (sq.ft) and Number of Bedrooms as input features from the user. Use Multiple Linear Regression to build the prediction model.

Program: House Price Prediction Using Multiple Linear Regression

This program uses **Area (sq.ft.)** and **Number of Bedrooms** as input features and predicts the **House Price** using Multiple Linear Regression.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Load the dataset
df = pd.read_csv("house.csv")

# Display the dataset
print("House Dataset:")
print(df)

# Select input features
X = df[["Area", "Bedrooms"]]

# Select target variable
y = df["Price"]

# Create Multiple Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Accept input from user
area = float(input("Enter house area in sq.ft: "))
bedrooms = int(input("Enter number of bedrooms: "))

# Create input data for prediction
new_house = pd.DataFrame({
    "Area": [area],
    "Bedrooms": [bedrooms]
```

```
})

# Predict house price
predicted_price = model.predict(new_house)

# Display predicted price
print("Predicted House Price:", predicted_price[0])
```

Sample `house.csv`

```
Area,Bedrooms,Price
1000,2,3000000
1200,2,3500000
1500,3,4500000
1800,3,5500000
2000,4,6500000
2500,4,8000000
```

Sample Output

```
House Dataset:
   Area  Bedrooms    Price
0  1000         2  3000000
1  1200         2  3500000
2  1500         3  4500000
3  1800         3  5500000
4  2000         4  6500000
5  2500         4  8000000

Enter house area in sq.ft: 1600
Enter number of bedrooms: 3

Predicted House Price: 4800000.0
```

Slip 23

Q1) A Write a python program to predict if a Person is Eligible to Vote (Age >= 18) using simple Logistic Classification. Accept person age from user. [10 marks]

Program: Vote Eligibility Using Logistic Regression

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Training data
age = np.array([[10], [15], [17], [18], [20], [25],
[30], [40]])

# 0 = Not Eligible, 1 = Eligible
eligible = np.array([0, 0, 0, 1, 1, 1, 1, 1])

# Create Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(age, eligible)

# Accept age from user
person_age = int(input("Enter person's age: "))

# Predict eligibility
prediction = model.predict([[person_age]])

# Display result
if prediction[0] == 1:
    print("Person is Eligible to Vote")
else:
    print("Person is Not Eligible to Vote")
```

Sample Output

```
Enter person's age: 20
Person is Eligible to Vote
```

Another example:

```
Enter person's age: 16
Person is Not Eligible to Vote
```

Q2) B Develop a program to perform different list methods such as add (append), extend, insert, remove and delete elements and display the updated list after each operation. [20 marks]

Program: Perform Different List Methods

```
# Create a list
my_list = [10, 20, 30, 40]

print("Original List:", my_list)

# 1. Append - Add an element at the end
my_list.append(50)
print("After append(50):", my_list)

# 2. Extend - Add multiple elements
my_list.extend([60, 70])
print("After extend([60, 70]):", my_list)

# 3. Insert - Add an element at a specific position
my_list.insert(2, 25)
print("After insert(2, 25):", my_list)

# 4. Remove - Remove a specific element
my_list.remove(30)
print("After remove(30):", my_list)

# 5. Delete - Delete an element using index
del my_list[1]
print("After deleting element at index 1:", my_list)
```

Sample Output

```
Original List: [10, 20, 30, 40]
After append(50): [10, 20, 30, 40, 50]
After extend([60, 70]): [10, 20, 30, 40, 50, 60, 70]
After insert(2, 25): [10, 20, 25, 30, 40, 50, 60, 70]
After remove(30): [10, 20, 25, 40, 50, 60, 70]
After deleting element at index 1: [10, 25, 40, 50,
60, 70]
```

OR

Q2) B Design a Model-Based Agent that maintains an internal state of soil moisture and weather conditions. Based on the current sensor readings and stored memory, the agent decides whether to water the plants. [20 marks]

Program: Model-Based Agent for Plant Watering

```
# Model-Based Agent for Plant Watering

# Internal state (memory)
internal_state = {
    "soil_moisture": "Unknown",
    "weather": "Unknown"
}

# Function to update internal state
def update_state(soil_moisture, weather):
    internal_state["soil_moisture"] = soil_moisture
    internal_state["weather"] = weather

# Function to decide the action
def decide_action():
    soil = internal_state["soil_moisture"]
    weather = internal_state["weather"]

    if soil == "Dry" and weather != "Rainy":
        return "Water the plants"
    elif soil == "Dry" and weather == "Rainy":
        return "Do not water - Rain expected"
    else:
        return "Do not water - Soil is moist"


# Accept current sensor readings
soil = input("Enter soil moisture (Dry/Moist): ")
weather = input("Enter weather condition
(Sunny/Cloudy/Rainy): ")

# Update internal state using current readings
update_state(soil, weather)

# Display internal state
print("\nInternal State:")
print("Soil Moisture:",
internal_state["soil_moisture"])
print("Weather:", internal_state["weather"])

# Decide and display action
action = decide_action()
```

```
print("Selected Action:", action)
```

Sample Output 1

```
Enter soil moisture (Dry/Moist): Dry
Enter weather condition (Sunny/Cloudy/Rainy): Sunny

Internal State:
Soil Moisture: Dry
Weather: Sunny
Selected Action: Water the plants
```

Sample Output 2

```
Enter soil moisture (Dry/Moist): Dry
Enter weather condition (Sunny/Cloudy/Rainy): Rainy

Internal State:
Soil Moisture: Dry
Weather: Rainy
Selected Action: Do not water - Rain expected
```

Sample Output 3

```
Enter soil moisture (Dry/Moist): Moist
Enter weather condition (Sunny/Cloudy/Rainy): Sunny

Internal State:
Soil Moisture: Moist
Weather: Sunny
Selected Action: Do not water - Soil is moist
```